Projeto Prático – 2026/2027

Gestão de Fábrica

Pretende-se um programa que faça a Simulação da Gestão (ao nível da domótica e sistema de controlo de motores) de uma dada fábrica. Esta fábrica tem vários tipos de utilizadores (pessoas com níveis de acesso diferentes e responsabilidades diferentes). Existem na fábrica vários tipos de motores (que é preciso monitorizar), e vários tipos de sensores, que permitem conhecer e manter um bom estado dos equipamentos e das instalações.

OBS: A solução do problema descrito neste enunciado pode ser aplicada em várias situações (Controlo de Escola, Centralina de um automóvel, Sistema de segurança/gestão de uma fábrica, etc.)

Os motores têm estados típicos: STOP, START, RUN, AVARIADO
Os motores quando em funcionamento, podem aquecer!. Considerem 3 classes de temperatura.
VERDE, AMARELO e VERMELHO.
VERDE: O motor está a funcionar bem!
AMARELO: O motor está a aquecer, deve ser enviado um aviso ao gestor;
VERMELHO: O motor está bem quente, pelo que deve ser avisado o Gestor e o motor para se proteger deve ficar no estado STOP.

Para tornar o problema mais próximo da realidade, a configuração da fábrica e de todo o conteúdo a gerir é dado num ficheiro em XML. Os intervalos de temperatura para cada uma das classes de temperatura é dada no ficheiro XML.

Considerações:
- A Fábrica tem uma hora de abertura e hora de encerramento;
- Quando existe algum alerta/aviso (da parte dos motores ou sensores) o gestor da fábrica deve ser logo informado, para desse modo atuar!;
- Cada dispositivo (SENSOR, MOTOR) está numa dada posição (X, Y) na fábrica.

Algumas definições dos dispositivos:

SENSOR (ID, MARCA, VALOR, PROBABILIDADE_AVARIA)

- A fábrica tem vários tipos de sensores (LUZ, HUMIDADE, FUMO, MÍSSEIS). Este sensor de **MÍSSEIS**, permite detetar, se um dado míssel vem na direção da fábrica….

MOTOR (ID, MARCA, CONSUMO_HORA, TEMPERATURA_AVISO, TEMPERATURA_PARAGEM, PROBABILIDADE_AVARIA)

- Todos os motores têm os métodos RUN; START; RESTART; STOP e ESTOU_QUENTE. Estes métodos podem ser re-implementados por cada um dos motores específicos.

USER (ID, NOME)
ADMIN (Pode fazer: ADD, REMOVER, LISTAR, RUN, MANUTENCAO)
NORMAL (Pode fazer: ADD, LISTAR, RUN, MANUTENCAO)
VISITANTE (Pode fazer: LISTAR)

Funcionalidades Gerais:

- Construtor da Fabrica, sendo dado o USER actual;

Fabrica::Fabrica(USER *ut)

- As configurações da Fábrica são dadas em ficheiro XML, com todas as informações;

bool Fabrica::Load(const string &ficheiro)

- O Simulador deve estar sempre a correr e quando se pretende introduzir alterações, deve

carregar-se na tecla 'M' de modo a aparecer um menu (nesse instante o simulador deve parar, até uma opção ser escolhida e executada!);

- Adicionar Utilizadores (ter em atenção as regras!);

  bool Fabrica::Add(User *ut)

- Adicionar Motores (Ver quem pode!);

  bool Fabrica::Add(Motor *m)

- Listar o estado atual da fábrica;

  void Fabrica::Listar(ostream &f = std::cout)

- Desligar um motor específico, dado o seu ID;

  void Fabrica::Desligar(int id_motor)

- Saber o estado de um dado Motor, dado o seu ID;

  ESTADO_MOTOR Fabrica::Get_Estado(int id_motor)

- Listar e devolver todos os Motores de um dado Tipo;

  List<Motor *> *Fabrica::Listar_Tipo(string Tipo, ostream &f = std::cout)

- Manutenção de todos os dispositivos (Motores e Sensores!), A manutenção de um motor poderá ser por exemplo alterar a temperatura para um valor (aleatório) dentro da temperatura VERDE do motor;

  bool Fabrica::Manutencao()

- Quais as Marcas de motores/sensores que mais avariam, deve devolver uma lista (ordenada) do que mais avaria para o mais fiável;

  List<string> *Fabrica::Ranking_Dos_Fracos()

- Quais motores que mais trabalharam, deve devolver uma lista (ordenada);

  List<Motor *> *Fabrica::Ranking_Dos_Mais_Trabalhadores()

- Enviar um relatório, em XML, do estado da fábrica. O relatório deverá ter a informação do total de consumo (do motor) nesse dia;

  void Fabrica::Relatorio(string fich_xml)

- Quando um sensor de Humidade envia um aviso, todos os motores que estão numa dada vizinhança (X) dele devem ser desligados!, o método deve devolver (por parametro) a lista dos Motores que foram desligados, também deve devolver o número de motores que foram desligados;

  int Fabrica::Aviso_Humidade(list<Motor *> &lm)

- Quando um sensor de FUMO envia um aviso, todos os motores devem ser desligados e deve ser invocado um video mostrando as zonas de saída da fábrica;

  int Fabrica::Aviso_Fumo(list<Motor *> &lm, string fich_video)

- No caso de o sensor de LUZ enviar um aviso (por exemplo quando a luminosidade for inferior a um dado valor!) deve passar um vídeo desejando um bom regresso a casa!, pois o tempo está a escurecer e aproxima-se a hora de dormir;

  int Fabrica::Aviso_Luz(string fich_video)

- No caso de o sensor de MÍSSEL enviar um aviso (por exemplo quando o valor for 1) deve passar um video mostrando os abrigos e a fábrica deve desligar-se completamente, guardando o estado atual!;

  void Fabrica::Aviso_Missel(string fvideo, string festado = "Estado.txt")

- Gravar um ficheiro que possa ser aberto e visualizado num browser com o estado atual da fábrica;

  void Fabrica::Listagem_HTML(string fich_html)

Observações:

- Podem e devem criar novos métodos auxiliares (privados) com os nomes que acharem por bem! E chamar nos métodos principais (públicos);
- Deverá ser usada a metodologia orientada a objetos, com a linguagem C++;
- A programação deve ser o mais standard possível.

Avaliação / Observações:

- Se a gestão da memória, não estiver correcta, haverá uma penalização de 5 valores. Devem implementar os destrutores convenientemente de modo a libertar explicitamente toda a memória alocada;
- É fornecido um **ficheiro XML** com uma possível configuração inicial de uma fábrica;
- É fornecido um **simulador** para visualização gráfica da fábrica;
- A instrução "auto" não deve ser usada;
- O código fonte deve ser devidamente comentado e acompanhado de documentação técnica. Podem optar por uma das seguintes modalidades: **Documentação gerada com o *Doxygen*** (ou ferramenta equivalente, com todas as classes, funções/métodos, estruturas de dados e ficheiros de cabeçalho documentados com comentários) **ou relatório técnico, em formato PDF** contendo, no mínimo, a descrição do problema e dos requisitos, a arquitetura e organização do código (classes e respetivas relações, incluindo diagrama UML), decisões de implementação mais relevantes, incluindo as estruturas de dados e os algoritmos escolhidos e a respetiva justificação, limitações conhecidas e possíveis melhorias;
- Se for detetado "copianço", trabalho anulado!
- Os grupos são constituídos por 2 a 3 alunos;
- Eventuais dúvidas serão esclarecidas pelos docentes da disciplina.

A avaliação do trabalho consiste em 2 etapas:

- **1ª Etapa, Avaliação em aula valendo 20% da nota prática, esta avaliação decorre na semana de 23 de novembro, durante as aulas TP/PL. Esta avaliação é válida para todas as épocas de avaliação!;**
- **2ª Etapa, Avaliação e Defesa, valendo os restantes 30% da nota prática.**

Entrega:

- A entrega para avaliação em **época normal** deve ser feita até **12.dez.2026**. A defesa ocorre no período das aulas de compensação.
- Para avaliação em **época de recurso**, serão adicionada algumas funcionalidades extra a este enunciado, e a entrega deve ser feita até **02.fev.2027**. A calendarização da defesa será divulgada oportunamente, de acordo com o mapa de exames de recurso.